Testing

Authentication Testing

1. Brute Force Protection / Account Lockout

Test Objective: Ensure the system prevents repeated login attempts to avoid brute force attacks.
Steps Performed:

1. Go to the login page.
2. Attempt to log in with an incorrect password multiple time (e.g., 5 times).
3. Observe the system's behaviour after the limit is reached.

Expected Result:
The account should be temporarily locked or otherwise protected, preventing further login attempts for a defined period.
Actual Result:
After 5 failed attempts, the system displayed:
"Too many failed attempts. Account locked 15 minutes."
Security logs also recorded LOGIN_FAILED events and an ACCOUNT_LOCKED event.
PASSED

2. Password Complexity Enforcement Test

Test Goal: Ensure only strong passwords are expected during registration
Steps Performed:

1. Click register in login page
2. Try to register with weak passwords:
   password → rejected
   12345678 → rejected
   abcdefghi → rejected
   Password1 → rejected
   Rutgerssas33@ → Accepted
3. Observe the error message.

Expected Result: Registration should be rejected with a clear error message describing the required complexity.
Actual Result:
All weak passwords were rejected. The system displayed:
"Password too weak! Must be 12+ chars, include uppercase, lowercase, number, and special char."
PASSED

3. Session Management

Test Goal: Session Security
Session Token Security observation

Session_token cookie is set with:

1. HttpOnly: prevents access via JavaScript
2. Secure: ensures transmission over HTTPS only

3. SameSite=Strict: prevents CSRF attacks
4. Expiry: 30 minutes from login

Session Expiry/Timeout

1. Test: Left the session idle and verified expiration.
2. Observations: Session automatically expires after 30 minutes.

Logout Handling

1. Test: Logged out and attempted to reuse session token.
2. Observations: Session token invalidated; access to protected pages denied.

Sessions are properly being stored in session.json and cleans it after logout.
PASSED

4. Logout Functionality

Test Description: Verify that logging out properly terminates the user session and prevents access to protected pages.
Steps Performed to check

1. Logged in with a valid user account.
2. Clicked the Logout button.
3. Checked cookies in the browser DevTools.
4. Attempted to access protected pages after logout.
5. Expiry: 30 minutes from login

Expected Result

1. The session_token cookie should be cleared or expired.
2. Accessing protected pages after logout should redirect to the login page.

Actual Result

1. Request Cookies before logout:

   session_token present with value mIE7iKTDqycdk6DtYmsMSAKS-gB9dITrwqmslrZIGW8

   Domain: 127.0.0.1, Path: /, Expires in 30 min, HttpOnly: ✓, Secure: ✓, SameSite: Strict
2. Response Cookies after logout:

   session_token cleared / empty

   Domain: 127.0.0.1, Path: /, Session cookie, HttpOnly: ✓, Secure: (still true), SameSite: Strict
3. Attempts to access /dashboard and other protected routes redirected to login page, confirming session termination.
4. PASSED

5. **Password Reset Security**

Test Description: Test the security of the password reset mechanism, including identity verification, OTP validation, expiration, and password strength enforcement.
Steps Performed:

1. Initiated password reset using valid username and email.
2. Received a 6-digit OTP via email.
3. Entered OTP within the allowed time (10 minutes) and reset password.

4. Tested invalid scenarios such as incorrect email or username, wrong OPT, expired OTP, and weak password.

Expected Result:

1. OTP should only be sent for valid username/email combinations.
2. Incorrect username/email should not proceed with reset.
3. OTP should be validated correctly and be time limited.
4. Expired or incorrect OTP should be rejected.
5. Password reset should only be allowed after successful OTP verification.
6. Valid error messages should be shown.

Actual Result:

1. Password reset should only be allowed after successful OTP verification.
2. Incorrect username/email returned: “Invalid username/email combination.”
3. Incorrect OTP returned: “Incorrect OTP. Try again.”
4. Expired OTP returned: “OTP expired. Please request a new one.”
5. When strong password is not entered during password reset, error message is displayed: “Password too weak. Must be 12+ chars with upper, lower, number, special.”

Conclusion: Password reset system is secure and well implemented.

PASSED

Authorization Testing

1. Horizontal Privilege Escalation

Test Goal: Test whether a user can access files belonging to another user without authorization.

Steps Performed:

1. Logged in as User1
2. Viewed available files in dashboard
3. Attempted to access files belonging to other users

Expected Result:

User should only access:

- Files they own
- Files explicitly shared with them

Actual Result:

User1 was only able to view:

- Their own uploaded files
- Files shared with them by other users

No access to other users’ private files was possible.

Conclusion: Proper access control mechanisms prevent horizontal privilege escalation by enforcing ownership and sharing rules.

PASSED

2. Vertical Privilege Escalation

Test Goal: Test whether a normal user can gain administrative privileges.

Steps Performed:

1. Logged in as normal user
2. Verified that admin-specific UI elements were not visible like admin can see all the files uploaded by users, shared with other users, and manage users as well as guest.
3. Attempted to manually access admin endpoints (e.g., /admin/dashboard)

Expected Result: User should not access admin features or data.

Actual Result:

- Admin functionality was not visible in the UI
- Direct access to admin routes did not grant elevated privileges and it bring the user or guest back to where we were at the same page
- Only user-level functionality was accessible

Conclusion: The application correctly enforces role-based access control at both UI and backend levels, preventing vertical privilege escalation.

PASSED

3. Direct Object Reference

Test Goal: Test whether a user can access unauthorized resources by modifying object identifiers.

Steps Performed:

1. Logged in as a normal user
2. Accessed a valid file URL
3. Modified the file identifier in the URL to reference another user's file

Expected Result: Access should be denied unless the file is owned by or shared with the user.

Actual Result:

5. Access to unauthorized files was denied.
6. Only files owned by the user or explicitly shared were accessible

Conclusion: The application correctly validates ownership and sharing permissions, preventing direct object reference vulnerabilities.

PASSED

4. Forced Browsing

Test Goal: Test whether users can access restricted pages by directly entering URLs.

Steps Performed:

1. Logged in as a normal user
2. Attempted to access restricted admin URLs (e.g., /admin/dashboard, /admin/users) directly via browser

Expected Result: User should not be able to access restricted admin pages.

Actual Result:

- Access to admin pages was blocked
- The application did not load the requested admin pages
- The user remained on the current page with no access to restricted content

Conclusion: The application effectively prevents forced browsing by enforcing access control at the backend and blocking unauthorized route access.
PASSED

Input Validation Testing

Fields Tested:

- Username during registration and login
- Tried to create a file name with a script

Test Input:<script>alert('XSS') </script>
Steps Performed:

1. Entered the above input in the username field.
2. Tried to submit the form.
3. Checked if the input appeared anywhere else in the app (like the dashboard or profile).
4. Tried similar tests with file names and displayed text, noting that the system blocks < and > in file names.

Observed Result: The input was rejected. Error message: "Invalid username! 3-20 chars, alphanumeric + underscore."

- File names with < and > could not be created.
- All text shown on the dashboard or other pages was displayed safely as plain text.

Impact:

- No XSS attacks are possible through usernames or other inputs.
- Input validation stops scripts from running.

Conclusion:

- The app safely blocks dangerous inputs.
- Different cases like file names or displayed text, are also safe.

PASSED

2. Path Traversal

Test Description: Tested the application for path traversal vulnerabilities by manipulating file paths in application endpoints to attempts unauthorized access to server-side files.
Steps performed:

1. Logged into the application and navigated to the dashboard.
2. Identified a valid file download link: https://127.0.0.1:5000/download/ML26PracticeMidterm.pdf
3. Modified the URL by replacing the filename with a directory traversal payload: https://127.0.0.1:5000/download/../app.py
4. I received response: Not Found (404)

Tested again

1. I repeated the test on another endpoint: https://127.0.0.1:5000/share/ML26PracticeMidterm.pdf
2. Modified the URL again: https://127.0.0.1:5000/share/../app.py
3. I received response: Not Found (404)

Expected Result: The application should reject any attempt to access files outside the designated directory and prevent traversal using ../.
Actual Result: Both modified requests returned 404 Not Found. No access to internal files such as app.py was granted. The application did not expose any sensitive data. Which mean knowing the URL is not enough.
Conclusion: The application enforces proper file path validation and successfully mitigates path traversal attacks across multiple endpoints.
PASSED

3. Command Injection

Test Description: Tested the application for command injection vulnerabilities by inserting malicious command payloads into user input fields.
Steps Performed:

1. Navigated to login or register page
2. Entered command injection payloads in username field:
   Test; ls
   Admin && whoami
   cbc; cat app.py
3. Submitted the forms

Expected Result: Application should reject malicious inputs and not execute system commands.
Actual Result: All inputs were rejected due to validation rules. No command execution occurred. No system-level data was exposed
Conclusion: The application properly validates user input and does not execute system commands, preventing command injection attacks.
PASSED

4. File Upload Vulnerabilities

Test Description: Tested the file upload functionality to ensure that only safe and allowed file types can be uploaded, and that malicious file types are properly rejected.
Steps Performed:

1. Navigated to the file upload feature through user/admin
2. Attempted to upload multiple file types: test.exe, test.py, test.html, test.js
3. Observed the application response for each upload attempt.
4. Uploaded a valid file type: test.pdf

Expected Result: The application should only allow safe file types (e.g.: txt, pdf, docx, xlsx, png, jpg, jpeg) and reject potentially dangerous file types.
Actual Result: Upload attempts for .exe, .py, .html, .js files were rejected with the message: "File type not allowed"

test.exe → File type not allowed
test.py → File type not allowed
test.html → File type not allowed
test.js → File type not allowed

Uploading a .pdf file was successful.

test.pdf → Upload successful

Conclusion: The application enforces strict file type validation and successfully prevents the upload of potentially malicious file.
PASSED

Session Security Testing

1. Session Fixation

Test Description: Tested whether the application is vulnerable to session fixation by verifying if session tokens are reused across login sessions.
Steps Performed:

1. Logged into the application and captured the session token: session_token=DRZonu57Q9EM2hef5BYOB-VabQb5UrRRfyaa1UlbKIs
2. Opened a private browser session and logged in again.
3. Captured the new session token: session_token=5foGW52Gw5wtG8DUD-kaOei0_iOOCtzUkfwlzC_LJaI
4. Compared both session tokens.

Expected Result: A new token should be generated after each login.
Actual Result: Different session tokens were generated for each login session. Session IDs were not reused.
Conclusion: The application regenerates session tokens upon login, preventing session fixation attacks.
PASSED

2. Session Hijacking

Test Description: Tested the application for session hijacking vulnerabilities by verifying the security attributes of session cookies.
Steps Performed:

1. Logged into the application
2. Opened Developer Tools (Ctrl+Shift+I)
3. Navigated to Network Tab → cookies.
4. Inspected the session_token cookie.

Expected Result: Session cookies should include security flags such as HttpOnly, Secure, and SameSite to prevent unauthorized access.

Actual Result: The session cookie contained the following security attributes:

HttpOnly: Enabled

Secure: Enabled

SameSite: Strict

Conclusion: The application properly secures session cookies, significantly reducing the risk of session hijacking.

PASSED

3. Concurrent Session Handling

Test Description: Tested whether the application properly handles multiple simultaneous sessions for the same user.

Steps Performed:

1. Logged into the application using a normal browser window (Brave).
2. Opened a private browser window (private brave window).
3. Logged into the application again using the same user credentials.
4. Performed actions in both sessions simultaneously.

Expected Result: The application should handle multiple sessions securely without session interference or data leakage.

Actual Result: Both sessions remained active simultaneously. Each session had a unique session token. No interference or data leakage was observed.

Conclusion: The application properly manages multiple sessions for the same user without compromising security.

PASSED

4. Session Timeout

Test Description: Tested whether user sessions expire after a defined period of inactivity.

Steps Performed:

1. Logged into the application.
2. Observed session cookie configuration (Max-Age=1800)
3. Allowed the session to remain idle.
4. Attempted to access the application after the timeout period (30 minutes).

Expected Result: The session should expire after the defined timeout, requiring the user to log in again.

Actual Result: Session expiration is configured for 30 minutes. After timeout, user is required to re-authenticate. Basically, bringing back to login page to open credentials and log in again. The session timeout reset back to 30 minutes.

Conclusion: The application enforces session expiration after inactivity, improving overall session security.

PASSED

Configuration Testing

1. Security Headers

Test Description: Check if the application implements recommended HTTP security headers.

Steps Performed:

1. Opened app in browser
2. Checked response headers in DevTools → Network tab → dashboard
3. Clicked the main request and inspected Response Headers.

Expected Result: All relevant security headers are present to mitigate common attacks.

Actual Result: All critical security headers are present:

Header	**Value**	**Status**
Content-Security-Policy	Default-src 'self';	Mitigates XSS
X-Frame-Options	DENY	Prevents clickjacking
X-Content-Type-Options	nosniff	Prevents MIME sniffing
X-XSS-Protection	1; mode=block	Legacy XSS protection
Strict-Transport-Security	max-age=31536000; includeSubDomains	Forces HTTPS
Permissions-Policy	geolocation=(), microphone=(), camera=()	Limits browser API access
Referrer-Policy	strict-origin-when-cross-origin	Controls Referrer info
Set-Cookie	session-token=hlE4Kggko4Y7MMiPIcIk63pbpZs4rTH0rkUWuAzzXzA; Secure; HttpOnly; SameSite=Strict	Secure session cookie

Conclusion: The application is well-configured for HTTP security headers, reducing the risk of XSS, clickjacking, session theft, and other common web attacks. No misconfigurations were observed in this area.

PASSED

2**. TLS/SSL configuration**

Tested URL: https://127.0.0.1:5000/

Test Date: 28 Mar 2026

Steps Performed:

1. Opened the application in browser via https://127.0.0.1:5000/.
2. Clicked on the NotSecure warning to view certificate details.

3. Verified certificate information, including Common Name(CN), Organization (O), validity dates, and public key.
4. Confirmed that HTTPS/TLS connection is active and the browser negotiates encryption.

I found that

Finding	Description	Impact	Proof	Recommendation	Severity
Self-signed certificate	The site uses a self-signed certificate for HTTPS. Browser shows “Not Secure” warning.	Users see a warning, may not trust the site; cannot be used safely in production.	Certificate CN = localhost, Issuer = MyApp, valid from 18 Mar 2026 to 18 Mar 2027	Replace with a valid CA-signed certificate for production. Ensure TLS 1.2+ is enforced.	Medium

Conclusion: TLS/SSL is active, encrypting traffic between client and server, which protects session tokens and data in transit. However, the self-signed certificate triggers browser warnings and is unsuitable for production. For production deployment, a CA-signed certificate must be used to ensure user trust and compliance with security best practices.
PASSED

3**. Error Handling**

Test Description: Check how the application handles unexpected error, invalid URLs, and invalid input, and whether sensitive server information is exposed to the user.
Steps Performed:

1. Opened the application in the browser.
2. Navigated to non-existent page, e.g., https://127.0.0.1:5000/nonexistentpage.
3. Entered the invalid data in input fields like empty strings or special characters not allowed by the system.
4. Observed the response and error messages got.

Excepted Result: User sees generic messages like “404 Not Found” or “Invalid input.” No stack traces, server paths, database info, or debug info is exposed. Detailed error information should be recorded only in server-side logs.
Actual Result: Non-existent page displayed a standard error message: “Not Found The requested URL was not found on the server. If you entered the URL manually, please check your spelling and try again.”
Invalid inputs server returned a generic “Invalid input” message. It also provides information like what is the input requirements.
No sensitive server information or stack traces were visible to the user and server logs captured detailed error info internally in security.logs
Severity is low

Conclusion: Error handling is implemented correctly. No sensitive information is exposed to the client, and internal errors are logged securely for developer review.
PASSED

4. **Debug mode disabled**
Test Description: Verify that Flask debug mode is disabled to prevent sensitive information from being exposed through error pages or stack traces.
Steps Performed:

1. Confirmed app.run(debug=False) in app.py
2. Access a non-existent URL: https://127.0.0.1:5000/nonexistentpage
3. Access a existent URL: https://127.0.0.1:5000/share/ques2img.png
4. Observe the error page received in response.

Expected Result: Normal Not Found, or 404 Not Found, or 500 Internal Server Error page, or application created error page. No stack trace, source code, or system information visible.
Actual Result: Accessing the non-existing page give me this error message: "Not Found The requested URL was not found on the server. If you entered the URL manually, please check your spelling and try again."
Accessing the existing page but do not have access to it like getting a from admin side and trying to access from the user side gives this error: "Access denied: only owner or admin can share this file."
Conclusion: No sensitive information, source code, or debug data is exposed. Severity is low.
PASSED

EXECUTIVE SUMMARY

This report documents a comprehensive security assessment of the web application hosted at https://127.0.1:5000/. The testing checked how safe the app is for users and their data. We tested login, password rules, user access, file uploads, sessions, and general system setting. The app handled all tests very well: accounts lock after too many wrong logins, strong passwords are required, sessions expire correctly, and users cannot access other users' files or admin areas. Input fields and file uploads are protected against attacks like XSS, command injection, and path traversal. Security headers are properly set, error messages do not reveal sensitive information, and debug mode is turned off. Self-signed TLS certificate is used. Overall, the application is secure.

Testing Methodology

Tools and techniques used:

1. Manual testing: Browser (Brave), private incognito sessions, URL manipulation for forced browsing and direct object reference rests.
2. Developer Tools: Network/console inspection for cookies, HTTP headers, and session tokens.
3. Security headers check: Using browser DevTools to verify HTTP security headers.
4. Payload testing: Manual insertion of XSS (<script>alert('XSS') </script>), command injection (; ls, && whoami), and path traversal payloads (…/app.py).

5. Password and session testing: attempted weak password, OTP expiry check, and concurrent sessions verification. These all have been tested manually in the application.

Findings

Critical

- **None observed.** All high-impact vulnerabilities (XSS, command injection, privilege escalation) were mitigated

High

- **Self-signed TLS certificate**
    - **The impact is like, the u**sers see browser warnings, not suitable for production.
    - **Steps:** Open https://127.0.0.1:5000/, inspect certificate.
    - **Solution:** We can obtain a valid CA-signed certificate, enforce TLS 1.2+ or 1.3.
    - **Severity:** Medium

Medium

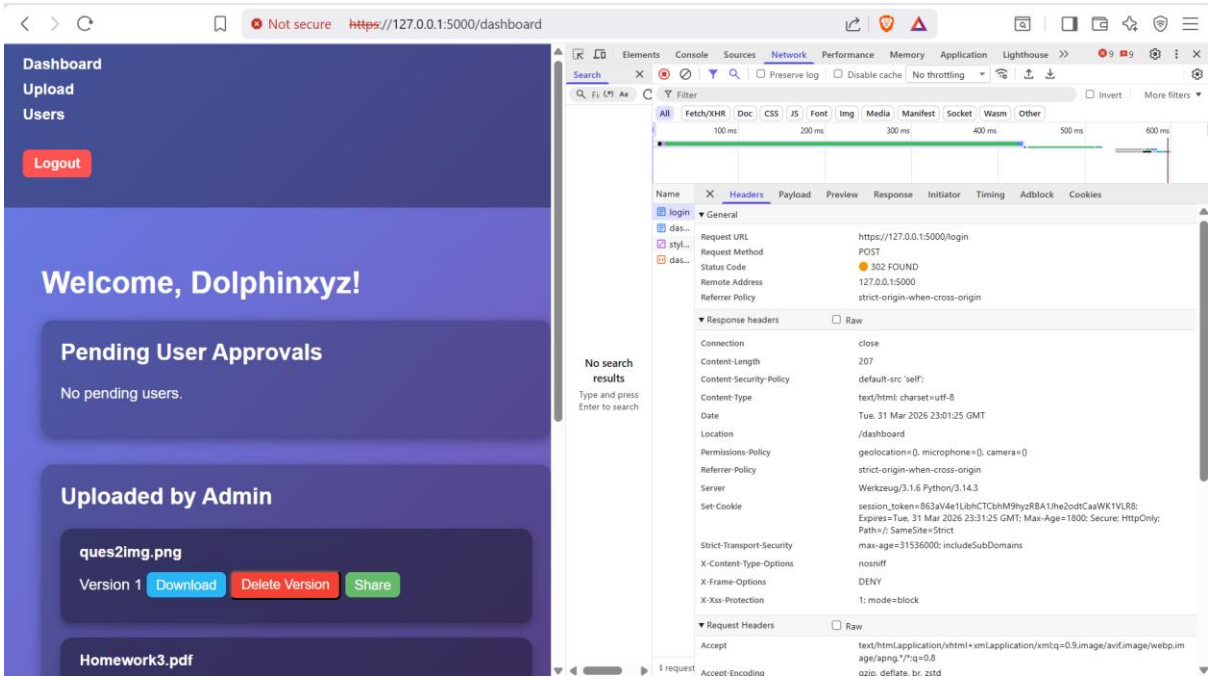
- **Password reset OTP security**: All tests passed, no gaps found.

Low

- **Error handling**: Observed generic error messages, no stack traces or sensitive info exposed.
- **Debug mode**: disabled in app.py code

Configuration Testing

I have tested security headers in browser DevTools



Acknowledgement

For this project, Google Gemini was just for some assistance. Gemini was used to help generate some code, assist with testing strategies, and support code cleanup and refinement. All outputs were reviewed, modified, and debugging.